

VITRYNES

Marketplace de Moda Regional com Delivery

Diagrama de Classes

Artefato de Modelagem e Projeto — UML

Data: 16/06/2026

1. Sumário

1. Sumário	2
2. Identificação	4
3. Objetivo	4
4. Definições e Siglas	4
5. Visão Geral do Diagrama	6
6. Convenções e Notação UML Adotadas	7
6.1. Estrutura das Classes	7
6.2. Visibilidade	7
6.3. Relacionamentos	7
7. Classes por Domínio	8
7.1. Identidade e Autenticação	8
Usuario	8
Cliente	8
Lojista	9
Entregador	9
7.2. Endereçamento	10
Endereco	10
7.3. Catálogo e Estoque	11
Loja	11
Produto	12
Estoque	13
7.4. Carrinho de Compras	13
CarrinhoCompras	13
ItemCarrinho	14
7.5. Pedidos	14
Pedido	14
ItemPedido	15
7.6. Pagamento e Financeiro	15
Pagamento	15
Cartao	16
ContaBancaria	17
7.7. Logística e Entrega	17
Entrega	17
ProblemaEntrega	18
7.8. Suporte e Engajamento	18
Notificacao	18
Avaliacao	19
8. Relacionamentos Principais	20
9. Considerações Finais	21

2. Identificação

Projeto	Vitryne — Marketplace de Moda Regional com Delivery
Organização	Vitryne
Equipe	Bruno Abraham · Gabriel Rodrigues · Henrique Pacheco · Hugo Zuin · João Ehlers · Leonardo Xavier
Autores	Gabriel Rodrigues, Leonardo Xavier e Igor Costa
Data de Criação	27/05/2026
Última Revisão	16/06/2026

3. Objetivo

Este documento apresenta o Diagrama de Classes (UML) elaborado na etapa de Modelagem e Projeto da plataforma Vitryne. Destina-se à equipe de desenvolvimento e aos revisores acadêmicos, comunicando a estrutura orientada a objetos do sistema — classes, atributos, métodos e relacionamentos — que servirá de base para a implementação das entidades JPA, dos serviços de domínio e dos contratos de API. O artefato documenta cada classe individualmente com suas responsabilidades, formaliza a especialização de Usuario nos três perfis de ator e mapeia os relacionamentos com suas cardinalidades.

4. Definições e Siglas

Glossário dos termos técnicos e siglas utilizados neste documento, em ordem alfabética.

TERMO / SIGLA	DEFINIÇÃO
Agregação	Relacionamento em que uma classe contém outra, mas as instâncias têm ciclo de vida independente.
Associação	Relacionamento estrutural entre duas classes que indica que uma usa ou conhece a outra.
Atributo	Variável de estado pertencente a uma classe (precedida por sinal '-' indica privado).
Cardinalidade	Número de instâncias envolvidas em um relacionamento (1, 0..1, 0..*, 1..*, etc.).
Classe	Estrutura que define atributos e métodos de objetos do domínio.
Composição	Relacionamento em que a existência da parte depende do todo (ciclo de vida acoplado).
DTO	Data Transfer Object — objeto usado para transportar dados entre camadas.
Especialização	Relacionamento de herança entre uma superclasse e suas subclasses.
Generalização	Relacionamento UML que indica que uma classe é uma forma genérica de outras (ISA).

TERMO / SIGLA	DEFINIÇÃO
Método	Operação ou comportamento exposto por uma classe (precedido por sinal '+' indica público).
MVC	Model-View-Controller — padrão arquitetural que separa responsabilidades.
OO	Orientação a Objetos — paradigma baseado em classes e objetos.
POJO	Plain Old Java Object — objeto Java simples, sem dependências de frameworks.
UML	Unified Modeling Language — linguagem padrão de modelagem de software.
Visibilidade	Modificador de acesso: '+' público, '-' privado, '#' protegido.

Tabela 1 — Glossário de termos e siglas.

6. Convenções e Notação UML Adotadas

Para garantir consistência na leitura e na interpretação do modelo, foram adotadas as convenções descritas a seguir.

6.1. Estrutura das Classes

- Cada classe é representada como um retângulo dividido em três compartimentos: nome, atributos e métodos.
- Atributos seguem a notação UML: visibilidade + nome + tipo (ex.: -nome: String).
- Métodos seguem a notação UML: visibilidade + nome(parâmetros) + tipo de retorno (ex.: +login(email, senha): String).

6.2. Visibilidade

- Sinal '+' indica visibilidade pública (acessível externamente).
- Sinal '-' indica visibilidade privada (acesso interno à classe).
- Sinal '#' indica visibilidade protegida (acessível a subclasses).

6.3. Relacionamentos

- Associação: linha sólida entre duas classes, com nome textual (ex.: "possui", "gerencia").
- Generalização (herança): seta com triângulo vazado apontando para a superclasse (Usuario).
- Cardinalidades: representadas nas extremidades das linhas (1, 0..1, 0..*, 1..*, etc.).
- Os relacionamentos são detalhados na Seção 8 com cardinalidade e significado semântico.

7. Classes por Domínio

Esta seção descreve cada uma das 19 classes do modelo, organizadas em oito domínios funcionais. Cada classe apresenta um breve contexto, seguida das tabelas completas de atributos e métodos.

7.1. Identidade e Autenticação

Agrupar a classe genérica de Usuario e suas três especializações (Cliente, Lojista e Entregador), responsáveis pela identificação e perfil de cada ator da plataforma.

Usuario

Classe genérica de autenticação compartilhada por todos os perfis.

Representa a base de autenticação e autorização da plataforma. Toda conta na Vitryne é instanciada como um Usuario e especializada em Cliente, Lojista ou Entregador conforme o atributo role.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único do usuário.
usuario	String	Nome de usuário (login).
email	String	E-mail único do usuário.
senhaHash	String	Hash BCrypt da senha.
role	String	Perfil: CONSUMER, STORE, DELIVERER, ADMIN.
criadoEm	DateTime	Data e hora de criação da conta.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
login(email, senha)	String	Autentica o usuário e retorna token JWT.
logout()	void	Invalida a sessão atual.
alterarSenha(senhaAtual, novaSenha)	void	Altera a senha mediante reautenticação.
recuperarSenha(email)	void	Inicia fluxo de recuperação de senha.

Cliente

Especialização de Usuario com dados do consumidor final.

Representa o consumidor que utiliza a plataforma para comprar produtos. Adiciona dados pessoais e operações de compra (carrinho, pedido, devolução).

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
nome	String	Nome completo do cliente.

ATRIBUTO	TIPO	DESCRIÇÃO
cpf	String	CPF do cliente, validado no cadastro.
rg	String	RG do cliente.
telefone	String	Telefone de contato.
dataNascimento	Date	Data de nascimento do cliente.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
adicionarAoCarrinho(produto, qtd)	void	Adiciona um item ao carrinho de compras.
realizarPedido()	Pedido	Confirma o checkout e cria um novo pedido.
solicitarDevolucao(pedido, motivo)	void	Solicita devolução de um pedido entregue.
visualizarHistorico()	List<Pedido>	Lista o histórico de pedidos do cliente.

Lojista

Especialização de Usuario com dados do gestor da loja.

Representa o proprietário ou gestor de uma loja parceira. Responsável pelo cadastro de produtos e pela gestão dos pedidos recebidos.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
nome	String	Nome completo do lojista.
cnpj	String	CNPJ da empresa associada ao lojista.
telefone	String	Telefone de contato.
dataNascimento	Date	Data de nascimento do lojista.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
confirmarPedido(pedidoId)	void	Confirma o atendimento de um pedido recebido.
recusarPedido(pedidoId, motivo)	void	Recusa um pedido informando o motivo.
cadastrarProduto(produto)	void	Cadastra um novo produto no catálogo da loja.
visualizarVendas()	List<Pedido>	Lista o histórico de vendas da loja.

Entregador

Especialização de Usuario com dados do parceiro logístico.

Representa o profissional autônomo responsável pela coleta, entrega e logística reversa. Possui controle de disponibilidade (ONLINE/OFFLINE) e dados do veículo utilizado.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
nome	String	Nome completo do entregador.
cpf	String	CPF do entregador.
cnh	String	Número da Carteira Nacional de Habilitação.
placaVeiculo	String	Placa do veículo utilizado nas entregas.
tipoVeiculo	String	Tipo do veículo (moto, carro, bicicleta, etc.).
telefone	String	Telefone de contato.
dataNascimento	Date	Data de nascimento do entregador.
disponivel	Boolean	Indica se o entregador está ONLINE para receber entregas.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
aceitarEntrega(entregaId)	void	Aceita uma entrega ofertada na fila.
recusarEntrega(entregaId)	void	Recusa uma entrega ofertada.
confirmarColeta(entregaId)	void	Confirma a coleta do produto na loja (com geofencing).
confirmarEntrega(entregaId)	void	Confirma a entrega ao consumidor (com geofencing).
alternarDisponibilidade()	void	Altera o status entre ONLINE e OFFLINE.

7.2. Endereçamento

Classe responsável pelo armazenamento estruturado de endereços, utilizada por clientes (entregas), lojistas (loja física) e pedidos (destino).

Endereco

Endereço estruturado com geolocalização.

Centraliza a representação de endereços na plataforma, permitindo geocodificação para cálculo de proximidade e roteirização. Pode ser marcado como padrão para usuários com múltiplos endereços.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único do endereço.
cep	String	CEP do endereço.
logradouro	String	Logradouro (rua, avenida, etc.).
numero	String	Número do imóvel.
complemento	String	Complemento (apto, bloco, etc.).

ATRIBUTO	TIPO	DESCRIÇÃO
bairro	String	Bairro do endereço.
cidade	String	Cidade do endereço.
estado	String	Estado do endereço.
latitude	Double	Coordenada de latitude geocodificada.
longitude	Double	Coordenada de longitude geocodificada.
padrao	Boolean	Indica se é o endereço padrão do usuário.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
validarCep()	Boolean	Valida o CEP via serviço externo.
geocodificar()	void	Obtém as coordenadas geográficas do endereço.

7.3. Catálogo e Estoque

Agrupar as classes que compõem o catálogo: a loja cadastrada, os produtos oferecidos e o controle de estoque por tamanho.

Loja

Vitrine digital de um lojista na plataforma.

Representa a loja parceira como entidade pública do marketplace. Controla status operacional (aberta/fechada/pausada), horário de funcionamento e o raio máximo de entrega.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único da loja.
nome	String	Nome fantasia da loja.
cnpj	String	CNPJ da loja.
descricao	String	Descrição pública da loja.
logoUrl	String	URL do logotipo da loja.
bannerUrl	String	URL do banner da loja.
status	String	Status atual: ABERTA, FECHADA, PAUSADA, SUSPENSA, INATIVA.
horarioFuncionamento	String	Horário configurado por dia da semana.
raioEntregaKm	Integer	Raio máximo de entrega em quilômetros.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
abrirLoja()	void	Abre a loja manualmente (sobrescreve horário).
fecharLoja()	void	Fecha a loja manualmente.
pausarLoja()	void	Coloca a loja em pausa temporária.
estaAberta()	Boolean	Retorna se a loja está aberta no momento atual.
calcularDistancia(lat, lng)	Double	Calcula a distância em km entre a loja e uma coordenada.

Produto

Item do catálogo cadastrado por um Lojista.

Representa uma peça de moda ofertada na plataforma, com atributos visuais, comerciais e de avaliação. Suporta preço promocional com período de vigência.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único do produto.
nome	String	Nome do produto.
descricao	String	Descrição detalhada do produto.
preco	Double	Preço de venda em reais.
precoPromocional	Double	Preço promocional ativo (opcional).
tipo	String	Categoria do produto (camiseta, calça, vestido, etc.).
cor	String	Cor do produto.
avaliacao	Float	Avaliação média atribuída pelos clientes (0 a 5).
status	String	Status do produto: ATIVO, INATIVO.
fotosUrls	List<String>	Lista de URLs das fotos do produto.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
verificarDisponibilidade()	Boolean	Verifica se há estoque disponível para o produto.
aplicarDesconto(percentual)	void	Aplica desconto percentual sobre o preço de venda.
calcularPrecoFinal()	Double	Retorna o preço final considerando promoções vigentes.

Estoque

Controle de quantidade em estoque por produto.

Representa o controle de estoque de um produto, referenciado por `produtoId`, permitindo o gerenciamento independente da quantidade disponível.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
<code>id</code>	Long	Identificador único do registro de estoque.
<code>produtoId</code>	Long	Identificador do produto ao qual o estoque pertence.
<code>quantidade</code>	Integer	Quantidade disponível em estoque.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
<code>diminuirEstoque(qtd)</code>	void	Decrementa o estoque após uma venda.
<code>aumentarEstoque(qtd)</code>	void	Incrementa o estoque (reposição ou devolução).
<code>estaDisponivel()</code>	Boolean	Retorna se há ao menos 1 unidade disponível.

7.4. Carrinho de Compras

Classes responsáveis pelo carrinho de compras do cliente, incluindo o carrinho em si e os itens nele contidos.

CarrinhoCompras

Carrinho ativo de um Cliente.

Representa o carrinho persistente de um cliente. Calcula a previsão de valor total dos itens antes do checkout e valida a disponibilidade dos produtos.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
<code>id</code>	Long	Identificador único do carrinho.
<code>previsaoValorTotal</code>	Double	Valor total estimado dos itens no carrinho.
<code>atualizadoEm</code>	DateTime	Data e hora da última alteração.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
<code>adicionarItem(produto, tamanho, qtd)</code>	void	Adiciona um item ao carrinho.
<code>removerItem(itemId)</code>	void	Remove um item do carrinho.
<code>atualizarQuantidade(itemId, qtd)</code>	void	Atualiza a quantidade de um item.
<code>calcularTotal()</code>	Double	Recalcula o valor total do carrinho.
<code>limpar()</code>	void	Esvazia o carrinho.
<code>validarDisponibilidade()</code>	List<ItemCarrinho>	Retorna itens que ficaram indisponíveis.

ItemCarrinho

Item individual dentro de um CarrinhoCompras.

Representa cada produto adicionado ao carrinho, com sua quantidade, preço unitário no momento da adição e tamanho selecionado.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único do item de carrinho.
quantidade	Integer	Quantidade do item no carrinho.
precoUnitario	Double	Preço unitário no momento da adição.
tamanho	String	Tamanho selecionado pelo cliente.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
calcularSubtotal()	Double	Retorna o subtotal do item (preço × quantidade).
validarEstoque()	Boolean	Valida se a quantidade ainda está disponível.

7.5. Pedidos

Classes responsáveis pelo ciclo de vida do pedido após o checkout, incluindo o pedido e seus itens.

Pedido

Pedido criado a partir da finalização do checkout.

Representa um pedido confirmado. Possui status que percorre o ciclo de vida (PENDENTE → CONFIRMADO → PRONTO PARA COLETA → EM ENTREGA → ENTREGUE), valor total e valor de frete separados.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único do pedido.
status	String	Status atual no ciclo de vida do pedido.
tempoEntregaMin	Integer	Tempo estimado de entrega em minutos.
valorTotal	Double	Valor total cobrado do cliente.
valorFrete	Double	Valor do frete cobrado.
criadoEm	DateTime	Data e hora da criação do pedido.
atualizadoEm	DateTime	Data e hora da última atualização de status.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
<code>confirmar()</code>	void	Marca o pedido como confirmado pelo lojista.
<code>cancelar(motivo)</code>	void	Cancela o pedido com o motivo informado.
<code>calcularTotal()</code>	Double	Recalcula o valor total do pedido.
<code>atualizarStatus(novoStatus)</code>	void	Avança o status para o próximo estado válido.
<code>podeSerCancelado()</code>	Boolean	Retorna se o pedido ainda pode ser cancelado.

ItemPedido

Item individual dentro de um Pedido.

Representa cada produto adquirido em um pedido, com quantidade, preço unitário registrado no momento da compra e tamanho selecionado.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
<code>id</code>	Long	Identificador único do item de pedido.
<code>quantidade</code>	Integer	Quantidade adquirida do produto.
<code>precoUnitario</code>	Double	Preço unitário no momento da compra.
<code>tamanho</code>	String	Tamanho adquirido do produto.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
<code>calcularSubtotal()</code>	Double	Retorna o subtotal do item (preço × quantidade).

7.6. Pagamento e Financeiro

Agrupa as classes que tratam do processamento de pagamentos e dos meios de recebimento associados (cartões dos clientes e conta bancária do lojista).

Pagamento

Registro do processamento de um pagamento via gateway.

Armazena o resultado do processamento de pagamento via gateway externo. Cada pedido possui um Pagamento associado com status, método utilizado e identificador da transação no gateway.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
<code>id</code>	Long	Identificador único do pagamento.
<code>valor</code>	Double	Valor processado pelo gateway.
<code>status</code>	String	Status: PENDENTE, APROVADO, RECUSADO, ESTORNADO.

ATRIBUTO	TIPO	DESCRIÇÃO
dataHora	DateTime	Data e hora do processamento.
metodo	String	Método utilizado (CARTAO_CREDITO, CARTAO_DEBITO, PIX, BOLETO).
transacaoId	String	Identificador da transação no gateway.
gateway	String	Gateway utilizado (Mercado Pago, Stripe, etc.).

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
processar()	void	Envia o pagamento para processamento no gateway.
estornar()	void	Solicita o estorno do pagamento.
confirmarCaptura()	void	Confirma a captura definitiva após aprovação.
consultarStatus()	String	Consulta o status atualizado no gateway.

Cartao

Meio de pagamento (cartão) salvo por um Cliente.

Representa um cartão tokenizado salvo pelo cliente para reutilização em compras futuras. Em produção, apenas o token retornado pelo gateway é persistido.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único do cartão.
numeroMascarado	String	Últimos dígitos visíveis do número do cartão.
titular	String	Nome impresso no cartão.
validade	String	Data de validade no formato MM/AA.
token	String	Token retornado pelo gateway para reutilização.
apelidoCartao	String	Apelido escolhido pelo cliente.
principal	Boolean	Indica se é o cartão padrão do cliente.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
validar()	Boolean	Valida os dados do cartão antes da tokenização.
tokenizar()	void	Tokeniza o cartão no gateway.

ContaBancaria

Dados bancários do Lojista para repasse.

Armazena a conta bancária do lojista utilizada pela plataforma para realizar o repasse das vendas após dedução da comissão.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único da conta bancária.
banco	String	Código ou nome do banco.
agencia	String	Número da agência bancária.
conta	String	Número da conta.
tipoConta	String	Tipo de conta (corrente, poupança, etc.).
titular	String	Nome do titular da conta.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
validarDados()	Boolean	Valida os dados bancários informados.

7.7. Logística e Entrega

Classes responsáveis pela execução logística do pedido: a entrega em si e os problemas eventualmente reportados durante o processo.

Entrega

Registro da execução logística de um Pedido.

Representa a execução de uma entrega pelo entregador, incluindo localização em tempo real, datas de retirada/entrega e validação por geofencing.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único da entrega.
localizacaoAtual	String	Descrição textual da localização atual.
latAtual	Double	Latitude atual do entregador.
lngAtual	Double	Longitude atual do entregador.
dtRetirada	Date	Data e hora da retirada na loja.
dtEntrega	Date	Data e hora da entrega ao consumidor.
dtEstimada	Date	Data e hora estimadas de entrega.
status	String	Status: PENDENTE, EM_COLETA, EM_ENTREGA, ENTREGUE.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
<code>confirmarColeta()</code>	void	Confirma a coleta do produto na loja.
<code>confirmarEntrega()</code>	void	Confirma a entrega ao consumidor.
<code>atualizarLocalizacao(lat, lng)</code>	void	Atualiza a posição atual do entregador.
<code>validarGeofencing(lat, lng, raioM)</code>	Boolean	Valida se está dentro do raio permitido.

ProblemaEntrega

Ocorrência reportada durante a entrega.

Registra problemas ocorridos durante a entrega (consumidor ausente, endereço não encontrado, produto danificado, etc.), permitindo rastreabilidade e ação por parte do suporte.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
<code>id</code>	Long	Identificador único do problema.
<code>tipo</code>	String	Tipo do problema (consumidor ausente, endereço, etc.).
<code>descricao</code>	String	Descrição textual do problema.
<code>dtRelato</code>	Date	Data e hora do relato.
<code>status</code>	String	Status: ABERTO, EM_TRATAMENTO, RESOLVIDO, ESCALADO.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
<code>registrar()</code>	void	Registra o problema no sistema.
<code>escalarParaAdmin()</code>	void	Escala o problema para a administração da plataforma.

7.8. Suporte e Engajamento

Classes auxiliares de apoio à experiência do usuário: notificações enviadas e avaliações registradas.

Notificacao

Notificação enviada a um Usuario.

Representa uma notificação enviada ao usuário (push, in-app, e-mail) sobre eventos relevantes: mudança de status de pedido, novas mensagens, promoções, etc.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
<code>id</code>	Long	Identificador único da notificação.
<code>titulo</code>	String	Título da notificação.

ATRIBUTO	TIPO	DESCRIÇÃO
mensagem	String	Conteúdo da mensagem.
tipo	String	Tipo: PEDIDO, ENTREGA, PROMOCAO, SISTEMA.
lida	Boolean	Indica se a notificação já foi lida.
enviadaEm	DateTime	Data e hora do envio.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
marcarComoLida()	void	Marca a notificação como lida pelo usuário.
enviar()	void	Envia a notificação pelos canais configurados.

Avaliacao

Avaliação registrada por um Cliente sobre Produto ou Loja.

Representa uma avaliação publicada pelo cliente após o recebimento do pedido, podendo ser direcionada ao produto ou à loja. Possui processo de moderação para conteúdo impróprio.

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único da avaliação.
nota	Integer	Nota atribuída (1 a 5).
comentario	String	Comentário textual do cliente.
criadaEm	DateTime	Data e hora da publicação.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
publicar()	void	Publica a avaliação após a entrega confirmada.
moderar()	void	Modera o conteúdo da avaliação (admin).

8. Relacionamentos Principais

A tabela a seguir consolida os relacionamentos entre as classes do modelo, especificando a cardinalidade e o significado semântico de cada associação.

RELACIONAMENTO	CARDINALIDADE	DESCRIÇÃO
Usuario → Cliente / Lojista / Entregador	ISA	Especialização (herança) de Usuario nos três perfis pelo atributo role.
Usuario ↔ Notificacao	1 : 0..*	Um usuário recebe diversas notificações ao longo do uso da plataforma.
Cliente ↔ Endereco	1 : 1..*	Um cliente pode ter um ou mais endereços cadastrados.
Cliente ↔ CarrinhoCompras	1 : 1	Cada cliente possui um único carrinho ativo.
Cliente ↔ Pedido (realiza)	1 : 0..*	Um cliente realiza diversos pedidos ao longo do tempo.
Cliente ↔ Cartao	1 : 0..*	Um cliente pode salvar vários cartões para uso futuro.
Cliente ↔ Avaliacao (faz)	1 : 0..*	Um cliente pode publicar várias avaliações após suas compras.
Lojista ↔ Loja (gerencia)	1 : 1..*	Um lojista gerencia uma ou mais lojas na plataforma.
Loja → Endereco (localizada em)	1 : 1	Cada loja possui um endereço físico associado.
Lojista ↔ ContaBancaria (possui)	1 : 0..1	Cada lojista possui uma conta bancária para recebimento dos repasses.
Loja ↔ Produto (cataloga)	1 : 0..*	Uma loja cataloga diversos produtos no marketplace.
Loja ↔ Pedido (recebe)	1 : 0..*	Uma loja recebe diversos pedidos de clientes.
Loja ↔ Avaliacao (recebe)	1 : 0..*	Uma loja pode receber avaliações dos clientes.
Produto ↔ Estoque (controla)	1 : 1..*	Um produto controla seu estoque de quantidade disponível.
Produto ↔ ItemCarrinho (referencia)	1 : 0..*	Um produto pode ser referenciado em vários itens de carrinho.
Produto ↔ ItemPedido (registra)	1 : 0..*	Um produto pode ser registrado em vários itens de pedido.
Produto ↔ Avaliacao (recebe)	1 : 0..*	Um produto pode receber avaliações dos clientes.
CarrinhoCompras ↔ ItemCarrinho (contém)	1 : 0..*	Um carrinho contém zero ou mais itens.
Pedido ↔ ItemPedido (contém)	1 : 1..*	Um pedido contém um ou mais itens.
Pedido ↔ Pagamento (possui)	1 : 1	Cada pedido possui um pagamento associado.
Pedido → Endereco (entregue em)	1 : 1	Cada pedido é entregue em um endereço informado pelo cliente.
Pedido ↔ Entrega (gera)	1 : 0..1	Cada pedido confirmado gera uma entrega para execução logística.
Pagamento → Cartao (pode usar)	0..1 : 0..1	Um pagamento pode utilizar um cartão tokenizado salvo pelo cliente.
Entregador ↔ Entrega (executa)	1 : 0..*	Um entregador executa diversas entregas.
Entrega ↔ ProblemaEntrega (pode ter)	1 : 0..*	Uma entrega pode ter zero ou mais problemas reportados.

Tabela 2 — Relacionamentos principais entre as classes do modelo.

9. Considerações Finais

O Diagrama de Classes apresentado nesta versão consolida o modelo orientado a objetos da plataforma Vitryne, cobrindo os domínios de identidade, catálogo, carrinho, pedidos, pagamento, logística e engajamento. O modelo serve de base direta para a implementação das entidades JPA no back-end Java/Spring Boot, para a definição dos DTOs de request/response e para a estruturação dos serviços de domínio.

Pontos a refinar nas próximas iterações: (i) padronizar o atributo de telefone em todas as classes que estendem Usuario, hoje duplicado nas três especializações; (ii) avaliar a extração de uma classe Pessoa abstrata para conter os atributos compartilhados (nome, telefone, dataNascimento) entre Cliente, Lojista e Entregador; (iii) considerar a inclusão de uma classe Devolucao explícita no modelo, atualmente representada apenas como método solicitarDevolucao() em Cliente; (iv) revisar a granularidade do método atualizarStatus() em Pedido para usar enum tipado em vez de String, garantindo segurança de tipos.